

Healthcare Clinic & Patient Appointment System

A Modular, Fault-Tolerant Clinical Scheduling & Triage Management System in Java

Course / Platform:	Object-Oriented Programming in Java (VITYarthi)	Date:	September 2026
Domain:	Healthcare Operations & Clinical Triage	Language:	Java SE 17+ (Tested on OpenJDK 21)
Architecture:	Three-Tier Layered / MVC-Lite Architecture	Persistence:	Multi-Entity CSV Storage (RFC 4180)

1. Introduction & Problem Statement

Outpatient clinics and healthcare centers handle extensive daily appointments, triage reviews, and clinical consultations. Manual record keeping or generic spreadsheets frequently lead to: (1) physician double-booking during identical time slots, (2) delayed emergency triage where critical patients wait behind routine checkups, (3) data loss upon program restart, and (4) runtime application crashes when malformed terminal input is entered.

Technical Solution: This project implements a fully executable, dependency-free Java CLI application that provides: demographic patient intake, multi-criteria clinical triage prioritization (CRITICAL → HIGH → MEDIUM → LOW), conflict-free appointment booking with doctor availability validation, automated billing invoice computation, and multi-entity CSV persistence.

2. Functional & Non-Functional Requirements

2.1 Three Core Functional Modules

Module	Feature Name	Functional Description
Module 1 Patient & Triage	Patient Intake & Triage	Registers patient demographic profiles (blood group, age) with triage classification (CRITICAL, HIGH, MEDIUM, LOW) and dynamic priority queue generation.
Module 2 Doctor Catalog	Staff & Specialty Catalog	Maintains doctor directory with consultation fees, working days, and keyword specialization filtering.
Module 3 Scheduling & Billing	Appointment & Invoicing	Conflict-free appointment booking (rejects doctor double-booking), visit completion tracking, and itemized billing invoices.
Persistence	Multi-CSV Synchronization	Automatic serialization of all entities to data/patients.csv, doctors.csv, and appointments.csv.

2.2 Non-Functional Requirements

- **Performance:** O(1) instantaneous lookups using in-memory LinkedHashMap primary key indexing.
- **Reliability:** Traps all NumberFormatException and stream EOF conditions without uncaught exceptions.
- **Referential Integrity:** Rejects appointment bookings for non-existent patients or doctors.
- **Zero Dependencies:** 100% Core Java SE (java.util, java.io, java.util.regex).

3. System Architecture & Object-Oriented Design

The project follows a Three-Tier Layered Architecture with clear separation of concerns across 10 modular classes and enums:

Class / Enum	Role / OOP Principle	Key Methods & Responsibilities
Person.java	Abstract Base Class	Encapsulates id, name, phone, email; declares abstract getRole().
Patient.java	Inheritance (Person)	Extends Person; adds age, gender, bloodGroup, and TriageSeverity.
Doctor.java	Inheritance (Person)	Extends Person; adds specialization, consultationFee, and availableDays.

Appointment.java	Domain Entity	Links patient, doctor, date, timeSlot, status, and clinical notes.
TriageSeverity.java	Enumeration	Defines LOW, MEDIUM, HIGH, CRITICAL with priority ranks (1-4).
StorageManager.java	Persistence Engine	Handles RFC 4180 CSV serialization for patients, doctors, and appointments.
ClinicService.java	Business Logic Layer	Triage priority queue sorting, conflict detection, and invoice computation.
ValidationUtils.java	Defensive Parser	Regex validators, stream-safe line parsing, date checks, and string truncation.
Main.java	Presentation / UI	Interactive 10-option CLI menu loop, table renderers, and input flow orchestration.

3.1 Class Structural Hierarchy



4. Core Algorithms & Implementation Details

1. Scheduling Conflict Avoidance Algorithm: Before persisting any appointment, ClinicService evaluates active bookings to detect physician overlaps:

```
public boolean hasSchedulingConflict(int doctorId, String date, String timeSlot) {
    return appointmentMap.values().stream()
        .filter(a -> a.getDoctorId() == doctorId)
        .filter(a -> a.getStatus() == AppointmentStatus.SCHEDULED)
        .filter(a -> a.getAppointmentDate().equalsIgnoreCase(date))
        .anyMatch(a -> a.getTimeSlot().equalsIgnoreCase(timeSlot));
}
```

2. Multi-Level Triage Queue Prioritization: Java streams sort patients according to clinical severity rank (CRITICAL = Rank 1, HIGH = Rank 2, MEDIUM = Rank 3, LOW = Rank 4), breaking ties via patient ID:

```
public List getTriagePriorityQueue() {
    return patientMap.values().stream()
        .sorted(Comparator.comparingInt((Patient p) -> p.getTriageSeverity().getPriorityRank())
            .thenComparing(Patient::getId))
        .collect(Collectors.toList());
}
```

5. Verification & Test Suite Matrix

Test ID	Test Scenario	Input Condition	Expected Outcome	Status
TC-01	Register Patient	Valid patient details	Stored & persisted to patients.csv	PASS
TC-02	Duplicate ID check	Existing patient ID 101	Rejected with ID collision error	PASS
TC-03	Triage Ordering	Mixed severity patients	CRITICAL sorted first ahead of LOW	PASS
TC-04	Doctor Search	Query: 'Cardio'	Returns Cardiology specialists only	PASS
TC-05	Conflict Booking	Same doctor, date, slot	Booking rejected; conflict prompt	PASS
TC-06	Foreign Key Check	Non-existent patient 999	Booking rejected; patient not found	PASS
TC-07	Invoice Calculation	Fee \$150 + Critical (\$50)	Total payable calculated as \$200.00	PASS
TC-08	CSV Persistence	Restart application	All records reloaded from data/ directory	PASS
TC-09	EOF Termination	Piped empty input stream	Graceful shutdown; data synced	PASS

6. Sample Execution Transcript

```
--- [3] Emergency Triage Priority Queue ---
+-----+-----+-----+-----+-----+
| ID   | Name           | Blood | Triage Level | Urgency Clinical Standing |
+-----+-----+-----+-----+-----+
| 101  | Eleanor Vance  | O+    | CRITICAL    | Immediate Emergency       |
| 102  | Marcus Brody   | A+    | HIGH        | Urgent / High Priority    |
| 103  | Sophia Chen    | B+    | MEDIUM     | Moderate / Needs Attention|
| 104  | Lucas Miller   | AB-   | LOW         | Routine / Non-urgent     |
+-----+-----+-----+-----+-----+

--- [9] Clinical Billing Invoice ---
Invoice Ref: INV-00501 | Apt ID: #501 | Patient: Eleanor Vance (CRITICAL)
Doctor: Dr. Sarah Jenkins (Cardiology) | Fee: $150.00 | Urgency Charge: $50.00
TOTAL AMOUNT PAYABLE: $200.00 | Payment Status: PENDING
```

7. Future Scope & Conclusion

Future Extensions: Integration with relational database engines (PostgreSQL / SQLite) via JDBC, graphical patient appointment calendars via JavaFX, and automated SMS appointment notifications.

Conclusion: The Healthcare Clinic & Patient Appointment System fulfills all requirements of the VITyarthi Build Your Own Project guidelines. It delivers 10 modular classes with clean inheritance hierarchies, multi-criteria stream triage sorting, robust CSV persistence, conflict-free appointment scheduling, and automated evaluation compatibility.